ספר פרויקט

משחק דמקה ב־Java ובּ־JavaFX

תיעוד תכנון, מימוש והפעלה

משחק רשת TCP, חשבונות ופרופיל משתמש

מגישים:

רביע אבו עאד — ת"ז 213984214

מותנא פרחאת — ת"ז 325079143

מנחה: ענבל רונן

אוקטובר 2026

תקציר ותוכן עניינים

הפרויקט הוא משחק דמקה ב־Java וב־JavaFX, הכולל משחק מול BOT ומשחק בין שני שחקנים דרך שרת TCP. השרת מנהל חדרים, מאמת מהלכים ושומר חשבונות ותוצאות ב־SQLite. לכל משתמש מחובר יש פרופיל המציג סטטיסטיקה והיסטוריית משחקים.

1. מבוא, מטרות ודרישות 3
2. כללי המשחק ומימוש מהלך 5
3. מבנה המערכת ואחריות המחלקות 6
4. תקשורת TCP וניהול חדרים 8
5. השחקן הממוחשב — BOT 11
6. חשבונות, SQLite ופרופיל 13
7. ממשק המשתמש ומסכי המערכת 15
8. בדיקות ואימות המערכת 17
9. הפעלה ומדריך משתמש 19
10. שימוש בבינה מלאכותית 20
11. תרומה אישית 21
12. סיכום, אתגרים ומקורות 22

1. מבוא, מטרות ודרישות

1.1 רקע ומטרת הפרויקט

דמקה היא משחק שבו ההחלטה על מהלך תלויה במיקום הכלים, באפשרויות האכילה ובתגובת היריב. מטרת הפרויקט היא ליצור יישום דמקה בעל ממשק גרפי, המאפשר משחק מול שחקן אחר באמצעות שרת ומשחק מול BOT. כל שחקן מפעיל את היישום במחשב שלו ומתחבר לחדר משותף.

1.2 מטרות

- להציג לוח ברור, בחירת כלי ומשבצת יעד, אנימציה ומשוב על מהלך שגוי.
- לאפשר יצירת חדר, שיתוף קוד, הצטרפות וסנכרון בין שני שחקנים.
- להוסיף הרשמה, התחברות ופרופיל עם סטטיסטיקה והיסטוריה שנשמרות לאורך זמן.

1.3 היקף

המערכת כוללת משחק בין שני שחקנים ברשת מקומית, משחק מול BOT ופרופיל אישי עם סטטיסטיקה והיסטוריה. משתמש מחובר יכול ליצור חדר או להצטרף לחדר. אפשר גם לתרגל מול BOT ללא חשבון, ללא רישום תוצאות בפרופיל.

1.4 תוצר וסביבת פיתוח

התוצר כולל יישום גרפי לשחקנים, שרת לניהול המשחק ומסד נתונים לשמירת הפרופילים. המשחק כתוב ב־Java, הממשק נבנה באמצעות JavaFX והבנייה מנוהלת ב־Gradle. Main מפעילה את יישום השחקן ו־DamkaServer מפעילה את השרת. העיצוב נמצא בקובץ damka.css.

1. מבוא, מטרות ודרישות — המשך

מגבלות המערכת

המימוש הנוכחי מכוון למשחק ברשת מקומית. כתובת השרת והפורט ניתנים להגדרה, אך תקשורת TCP אינה מוצפנת. שמירת הסיסמה כערך גיבוב במסד אינה מצפינה את ההעברה ברשת; Base64 בהודעות משמשת לייצוג טקסט.

לא קיים מנגנון לשחזור משחק פעיל לאחר ניתוק. כאשר השרת מזהה עזיבה או ניתוק במהלך משחק, הוא מסיים את המשחק לטובת היריב. לאחר התחברות מחדש אפשר לפתוח משחק נוסף.

משחקי BOT מתנהלים אצל הלקוח ותוצאותיהם מדווחות לשרת, ולכן מוצגות בנפרד מתוצאות משחקי הרשת. תוצאות אימון ללא התחברות או לאחר אובדן החיבור אינן נשמרות, ואין העלאה מאוחרת שלהן.

חיפוש ה־BOT מוגבל לעומק של רמת הקושי ולפונקציית ההערכה. לכן הוא אינו מבטיח בחירה מושלמת בכל מצב. חלון ההיסטוריה מציג את 100 התוצאות האחרונות, בעוד הסטטיסטיקות מחושבות מכל התוצאות.

2. כללי המשחק ומימוש מהלך

2.1 מצב פתיחה ותנועה

הלוח בגודל 8×8, עם 12 כלים לכל שחקן על המשבצות הכהות. X מתחיל בתחתית, נע מעלה ופותח את המשחק; O מתחיל למעלה ונע מטה. הקואורדינטות הן בטווח 0 עד 7. כלי רגיל נע משבצת אחת באלכסון קדימה, ואוכל בקפיצה של שתי משבצות מעל יריב אל יעד פנוי. כלי רגיל אוכל קדימה בלבד.

2.2 אכילה מחייבת והכתרה

אם קיימת אכילה, צעד רגיל אינו מותר. כאשר אפשר לאכול שוב אחרי קפיצה, ממשיכים עם אותו כלי ואותו שחקן נשאר בתור. כלי x בשורה 0 וכלי o בשורה 7 מוכתרים למלכים. מלך נע ואוכל לשני הכיוונים, אך אינו מעופף: צעד של משבצת אחת או קפיצת אכילה של שתי משבצות. ההכתרה נעשית לפני בדיקת ההמשך, ולכן מלך חדש עשוי להמשיך לאכול לאחור.

2.3 Game היא הסמכות לכללים

getValidMoves מחזירה את האפשרויות החוקיות: אכילות בלבד כאשר קיימת אכילה, או צעדים רגילים אחרת. באמצע רצף היא מגבילה את האפשרויות למיקום captureContinuationPosition. Game.move מחפשת את המקור והיעד ברשימה; מהלך שנדחה אינו משנה את הלוח או התור.

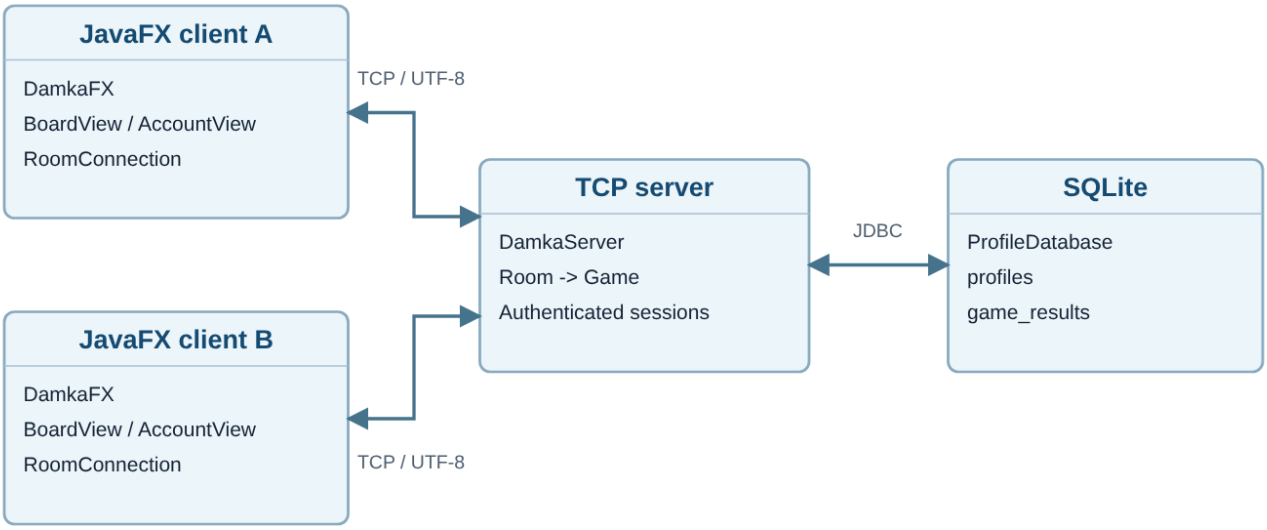
אחרי אישור Game מסירה כלי שנאכל, מזיזה את הכלי, מכתירה, שומרת lastMove ומשאירה חובת המשך או מחליפה תור. Game.Move כוללת מקור, יעד, סימן כלי ומיקום אכילה לצורך תצוגה. Board מבצעת גישה והזזה של הנתונים, בלי להחליט על חוקיות.

2.4 ניצחון

getWinnerSymbol מחזירה X או O כאשר ליריב אין מהלך חוקי, ו־'\0' כאשר אין מנצח. היעדר כלים גורם גם להיעדר מהלכים. פרישה, עזיבה וניתוק במשחק רשת מטופלים ב־Room, ולא כחוק תנועה של Game.

3. מבנה המערכת ואחריות המחלקות

3.1 שרת, לקוחות ושמירה



Server owns the online Game; clients display snapshots.

איור 1 — כל שחקן מפעיל לקוח; מצב המשחק המקוון והמסד נמצאים בשרת.

DamkaFX מנהלת קלט, מצב מסך ותגובות להודעות. BoardView מציירת ומנפישה, AccountView מציגה הזדהות ו־ProfileWindow מציגה פרופיל. RoomConnection מבצעת תקשורת ברקע. DamkaServer מקבלת לקוחות ומנתבת בקשות. Room מחזיקה משחק מקוון ושני משתתפים, ו־ProfileDatabase קוראת וכותבת SQLite.

3.2 שני מצבי משחק

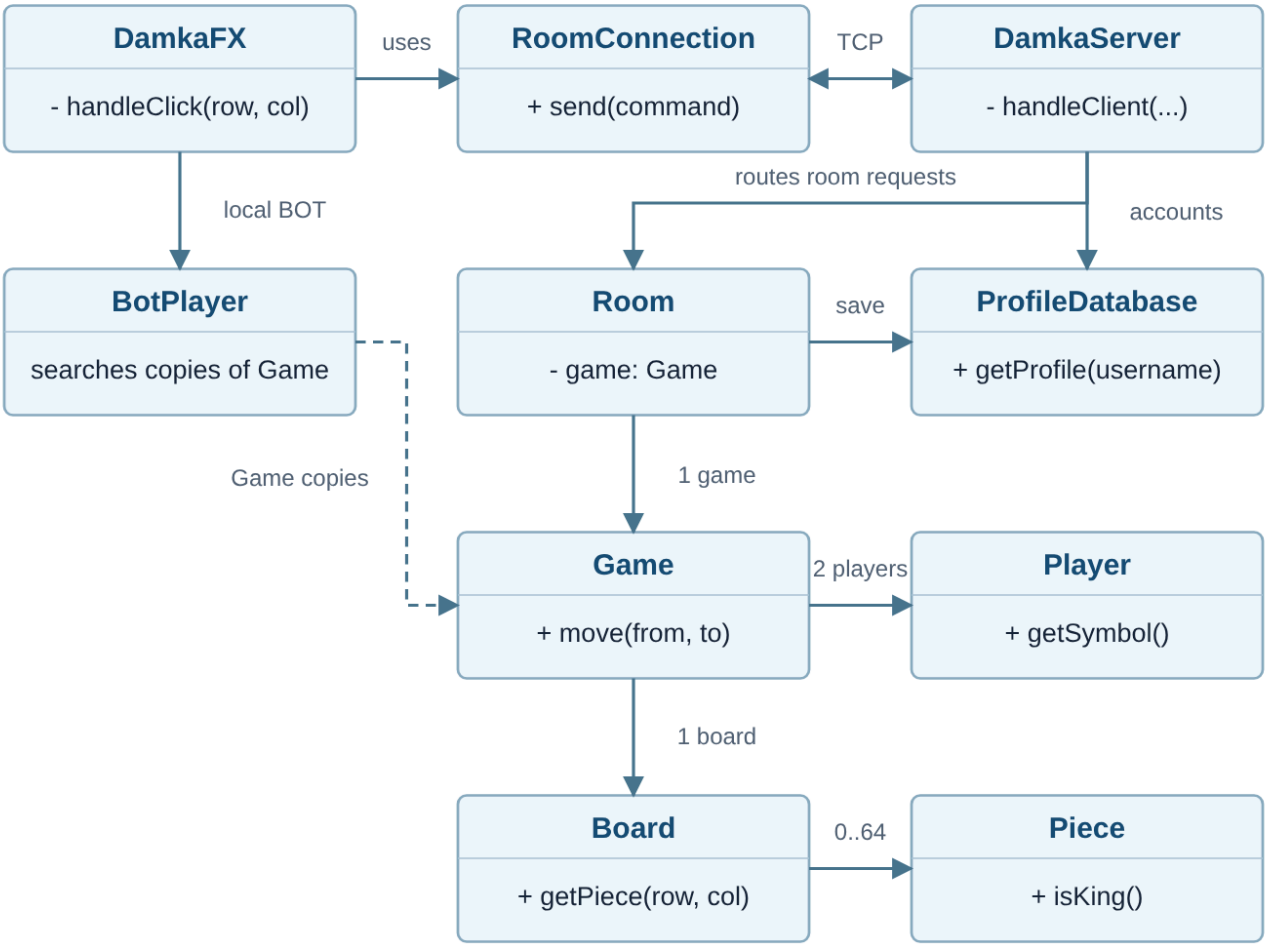
מול אדם אחר Game נמצאת בתוך Room בשרת, והלקוח מחזיק תמונת לוח לציור בלבד. מול BOT קיימת botGame מקומית ב־DamkaFX. BotPlayer מחפשת מהלך על עותקי Game. בשני המצבים משתמשים באותם כללים ובאותו BoardView.

3.3 הפרדה בין סוגי בדיקות

Game בודקת תנועה, אכילה, הכתרה וניצחון. Room בודקת מי שלח בקשה, חדר פעיל, תור ומבנה נתונים. הממשק בודק בחירה וזמינות פעולה. ההפרדה מונעת שכפול של כללי המשחק ומאפשרת בדיקה ללא חלון גרפי.

3. מבנה המערכת ואחריות המחלקות — המשך

3.4 תרשים מחלקות



איור 2 — הקשרים המרכזיים בין הממשק, התקשורת, המשחק והשמירה.

Position שומרת שורה ועמודה; Piece שומרת סימן, ואות גדולה מציינת מלך. Player היא שחקן במודל המשחק ואינה חשבון במסד. ProfileData היא רשומת נתוני פרופיל להעברה לתצוגה. החץ TCP מייצג קשר בין תהליכים, ולא קריאה ישירה למתודה על אובייקט משותף. התרשים מציג את הקשרים המרכזיים, ולא את כל מחלקות הספריות.

4. תקשורת TCP וניהול חדרים

4.1 חיבור השחקנים לשרת

ServerSocket ממתינה לחיבורים בשרת. accept מחזירה Socket עבור כל לקוח, והשרת מפעיל חוט ביצוע לטיפול בהודעותיו. הלקוח פותח חיבור בעת הרשמה או כניסה, ולאחר אימות החשבון משתמש בו גם לבקשות חדרים, מהלכים ופרופיל. כתובת השרת והפורט מוגדרים בהפעלה; פורט ברירת המחדל הוא 5000.

4.2 יצירת חדר והצטרפות

חדר מחבר בין שני משתתפים למשחק אחד. היוצר מקבל קוד בן שישה תווים ומשחק בתור X. שחקן אחר מזין את הקוד, מצטרף ומשחק בתור O. השרת מאשר את ההצטרפות, מודיע לשניהם שהחדר מוכן ושולח את לוח הפתיחה ואת התור הראשון.

בקשה או אירוע	תגובה מרכזית
CREATE_ROOM	ROOM_CREATED <code>, PLAYER X
JOIN_ROOM <code>	ROOM_JOINED <code>, PLAYER O
השחקן השני התקבל	ROOM_READY, BOARD, TURN X לשניהם
MOVE <fromRow> <fromCol> <toRow> <toCol>	MOVE_APPLIED, BOARD, TURN או ERROR
RESIGN / LEAVE_ROOM	GAME_OVER / ROOM_LEFT לפי מצב החדר

4.3 העברת מצב הלוח

מצב הלוח מועבר כמחרוזת של 64 סימנים לפי סדר השורות. נקודה מייצגת תא ריק, x/o כלי רגיל ו־X/O מלך. הלקוחות משתמשים במחרוזת כדי לצייר את הלוח. השרת הוא שקובע את מצב המשחק, ושני המשתתפים מקבלים ממנו את אותה תמונה.

4. תקשורת TCP וניהול חדרים — המשך

4.4 מסגרת ההודעות

TCP מעבירה זרם מסודר של בתים ואינה מגדירה בעצמה גבולות בין הודעות היישום. בפרויקט כל הודעה היא שורת טקסט בקידוד UTF-8. println שולחת שורה, ו־BufferedReader.readLine קוראת עד סוף השורה. לכן פקודה יכולה להגיע בכמה חלקים ברשת ועדיין להיקרא כהודעה שלמה. תחילת השורה, כגון MOVE או PROFILE_REQUEST, קובעת כיצד הבקשה תטופל.

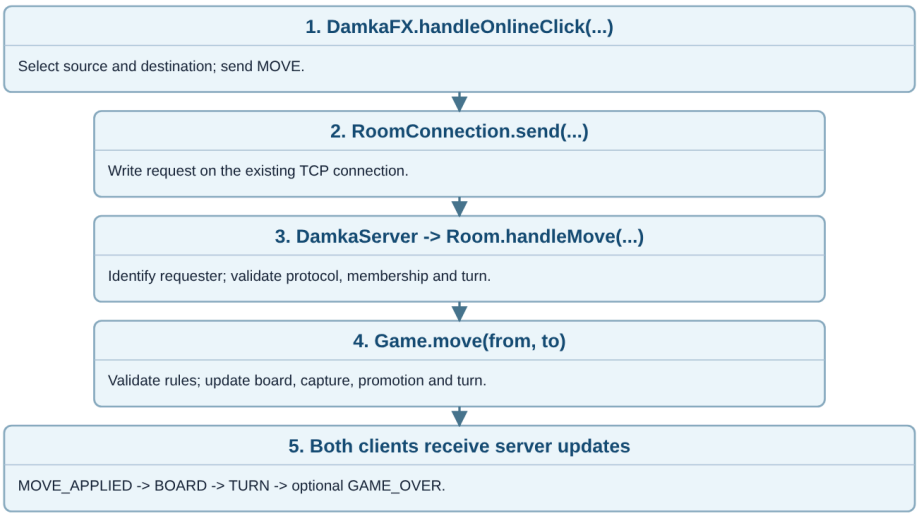
4.5 מעקב אחר מהלך

לדוגמה, שחקן X מבקש לעבור מהמשבצת (5,0) למשבצת (4,1) ושולח MOVE 5 0 4 1. התהליך מתחיל באירוע לחיצה ב־DamkaFX, עובר דרך RoomConnection אל DamkaServer ומגיע לחדר שאליו שייך החיבור. Room מזהה את השחקן ובודקת את מצב החדר ואת התור. Game.move בודקת את חוקי הדמקה ומעדכנת את המשחק רק כאשר המהלך חוקי.

שלב	אחריות ותוצאה
בחירת מקור ויעד	DamkaFX שולחת את בקשת המהלך דרך החיבור הקיים.
ניתוב הבקשה	DamkaServer מעבירה אותה ל־Room של השחקן.
בדיקת המהלך	Room בודקת את ההקשר ו־Game.move בודקת את הכללים.
דחיית בקשה	ERROR נשלחת למבקש; מצב הלוח והתור אינו משתנה.
קבלת בקשה	לשני הלקוחות נשלחות MOVE_APPLIED, BOARD ו־TURN, בסדר זה.
הצגה	הלקוח מנפיש את המהלך ואז מציג את תמונת השרת המעודכנת.

4. תקשורת TCP וניהול חדרים — המשך

4.6 מסלול המהלך



איור 3 — מסלול הבקשה והעדכון לשני הלקוחות.

עבודה במקביל ועדכון הממשק

RoomConnection קוראת הודעות בחוט רקע ומשתמשת בעובד כתיבה יחיד כדי לשמור על סדר הפקודות. פעולות הרשת עשויות להמתין, ולכן הן מבוצעות מחוץ לחוט הממשק.
Platform.runLater מוסיפה את עדכון המסך לתור העבודה של JavaFX. פעולות המשחק באותו חדר מסונכרנות, כדי שבדיקת מהלך, שינוי המצב ושליחת התוצאה יבוצעו כסדרה ולא יתערבבו עם מהלך נוסף.

4.7 תצוגה מסונכרנת

שני הלקוחות מקבלים את פרטי המהלך, את הלוח המעודכן ואת התור הבא. תחילה מוצגת אנימציית התנועה, ואחריה תמונת הלוח החדשה. כך שני השחקנים רואים את אותה התקדמות במשחק.

4.8 סיום משחק

המשחק מסתיים בניצחון לפי הכללים או בפרישת משתתף. עזיבה או ניתוק במהלך משחק פעיל מעניקים ניצחון ליריב. השרת שולח את התוצאה ושומר אותה בפרופילים. עזיבת חדר לפני הצטרפות שחקן שני אינה נרשמת כמשחק שהסתיים. לאחר הסיום ניתן לפתוח משחק נוסף.

5. השחקן הממוחשב — BOT

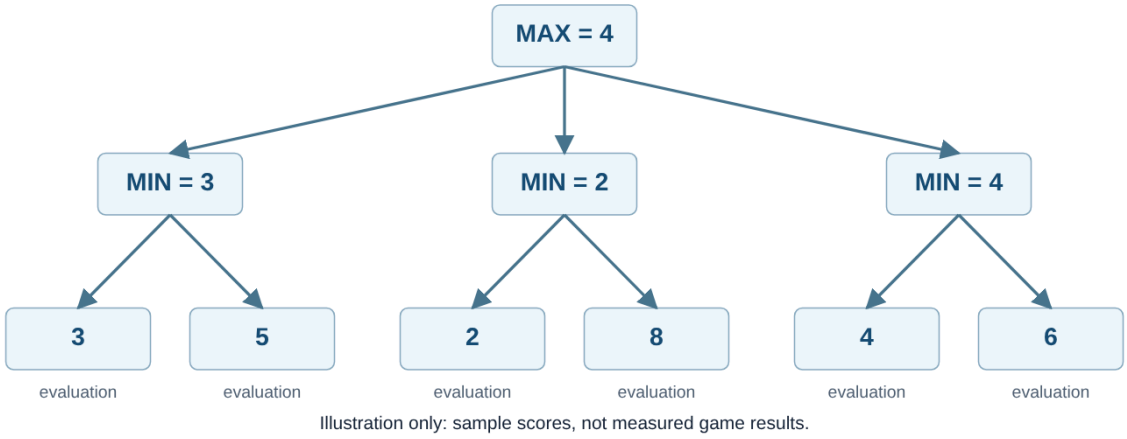
Minimax והעתקת מצב המשחק

ה־BOT ממומש במחלקה BotPlayer באמצעות Minimax עם גיזום Alpha-Beta. החיפוש בוחן מהלכים אפשריים ותגובות של היריב. המהלכים החוקיים מתקבלים מ־Game, ולכל אפשרות נוצר עותק עצמאי באמצעות new Game(game). המהלך הנבדק מתבצע על העותק, ולכן החיפוש אינו משנה את המשחק הפעיל.

הערכת הלוח ובחירה לפי התור

הציון מחושב מנקודת המבט של ה־BOT: כלי רגיל שווה 3 נקודות ומלך שווה 6. ההערכה מבוססת על ערך כלי ה־BOT פחות ערך כלי היריב. ניצחון והפסד מקבלים ציונים מכריעים. בתור ה־BOT נבחר הציון הגבוה ביותר; בתור היריב נבחר הנמוך ביותר, מתוך הנחה שהיריב יבחר את התגובה הטובה עבורו.

הבחירה בין הגדלת הציון להקטנתו נשענת על התור במצב המשחק הנבדק. באכילה מתמשכת אותו שחקן עשוי לקבל צעד נוסף, ולכן אין להחליף בין הצדדים באופן אוטומטי אחרי כל קפיצה.



איור 4 — דוגמה: MAX בוחרת את התוצאה 4 מתוך תוצאות MIN.

5. השחקן הממוחשב — BOT — המשך

עומק החיפוש ורמות הקושי

רמה	עומק	משמעות
Easy	1	בחינת צעד אחד קדימה והערכת המצב שהתקבל.
Medium	2	בחינת שני צעדים בעץ החיפוש.
Hard	3	בחינת שלושה צעדים בעץ החיפוש.

העומק נספר לפי צעד או קפיצה בודדת, ולא בהכרח לפי תור מלא. עומק גדול יותר בוחן המשכים נוספים ודורש יותר חישוב. הבחירה מבוססת על המצבים שנבדקו ועל פונקציית ההערכה.

דוגמה מספרית להחלטת Minimax

אפשרות של ה־BOT	ציוני תגובות היריב	הציון שהיריב יכפה
א’	3, −3	−3
ב’	0, 3	0

בדוגמה זו התור עובר ליריב לאחר בחירת האפשרות. היריב יבחר את הציון הנמוך בכל ענף, ולכן ה־BOT יעדיף את ב’: הציון 0 גבוה מ־−3. הדוגמה המספרית ממחישה כיצד תגובת היריב משפיעה על הבחירה.

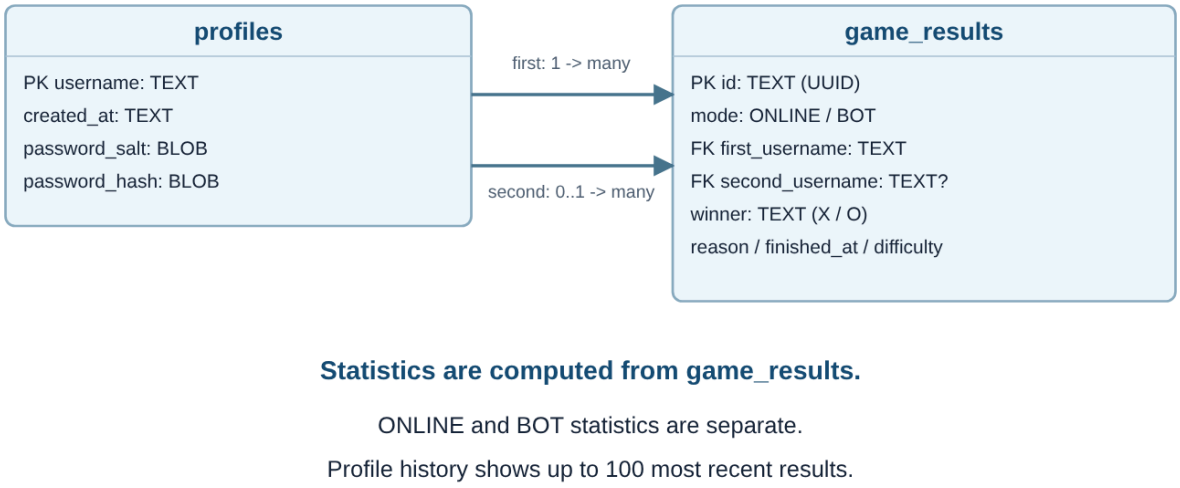
גיזום Alpha-Beta

Alpha-Beta שומרת גבולות לציונים שכבר נמצאו בחיפוש. כאשר המשך של ענף אינו יכול לשפר את הבחירה, החיפוש בענף נעצר. לדוגמה, אם כבר נמצא ענף שמבטיח ציון 0 וענף אחר מאפשר ליריב לכפות −3, אין צורך לבדוק את יתר תגובותיו כדי להעדיף את הענף הראשון. הגיזום מצמצם את מספר המצבים שנבדקים ושומר על ערך החלטת Minimax בעומק הנתון.

6. חשבונות, SQLite ופרופיל

6.1 נתונים מתמשכים בשרת

ProfileDatabase שומרת את החשבונות ואת תוצאות המשחקים במסד SQLite שנמצא במחשב השרת. הנתונים נשארים גם אחרי סגירת היישום והפעלה מחדש, ולכן המשתמש יכול לחזור לחשבון ולראות את הפרופיל שלו.



איור 5 — profiles מחזיקה חשבונות, ו־game_results מחזיקה תוצאות.

6.2 סכמה

טבלה	שדות עיקריים
profiles	username (PK), created_at, password_salt, password_hash
game_results	id (PK), mode, first_username / second_username (FK), winner, reason, finished_at, difficulty

6.3 הרשמה והתחברות

בהרשמה המשתמש בוחר שם וסיסמה ומאשר את הסיסמה. השרת בודק את הפרטים ויוצר חשבון כאשר השם פנוי. הסיסמה נשמרת בצורת hash ולא כטקסט המקורי. בעת כניסה השרת מאמת את הפרטים ומאפשר למשתמש לגשת למשחק ולפרופיל.

לאחר הכניסה כל פעולה בחיבור משויכת לחשבון שאומת. המשתמש יכול ליצור חדר, להצטרף למשחק ולצפות בתוצאות שלו. Sign out מסיימת את ההתחברות ומאפשרת להיכנס לחשבון אחר.

6. חשבונות, SQLite ופרופיל — המשך

6.4 שמירה פעם אחת

עם הצטרפות השחקן השני נוצר matchId מסוג UUID. בסיום משחק מקוון נשמרת שורה אחת עם המשתתפים, המנצח, הסיבה וזמן הסיום. מזהה id ייחודי מונע ספירה כפולה: חזרה על אותה תוצאה אינה מוסיפה משחק, ושימוש באותו id לתוצאה אחרת נדחה. חדר המתנה שנסגר אינו נרשם.

6.5 סטטיסטיקה והיסטוריה

getProfile קוראת את התוצאות שבהן החשבון הוא המשתתף הראשון או השני. לפי המושב ו־winner נקבע Win או Loss. Online ו־BOT נספרים בנפרד. Played, Wins ו־Losses כוללים את כל התוצאות; ההיסטוריה ממוינת מהחדש לישן ומציגה עד 100 משחקים. שניהם מחושבים מאותה קריאה למסד.

בקשה	תשובה ומשמעות
SIGNUP / LOGIN	AUTH_OK או AUTH_ERROR עבור פרטי החשבון.
PROFILE_REQUEST	PROFILE עם ProfileData מקודדת, או PROFILE_ERROR.
BOT_RESULT <id> <WIN|LOSS> <difficulty> <time>	BOT_SAVED או BOT_ERROR. זהות החשבון נלקחת מהחיבור.

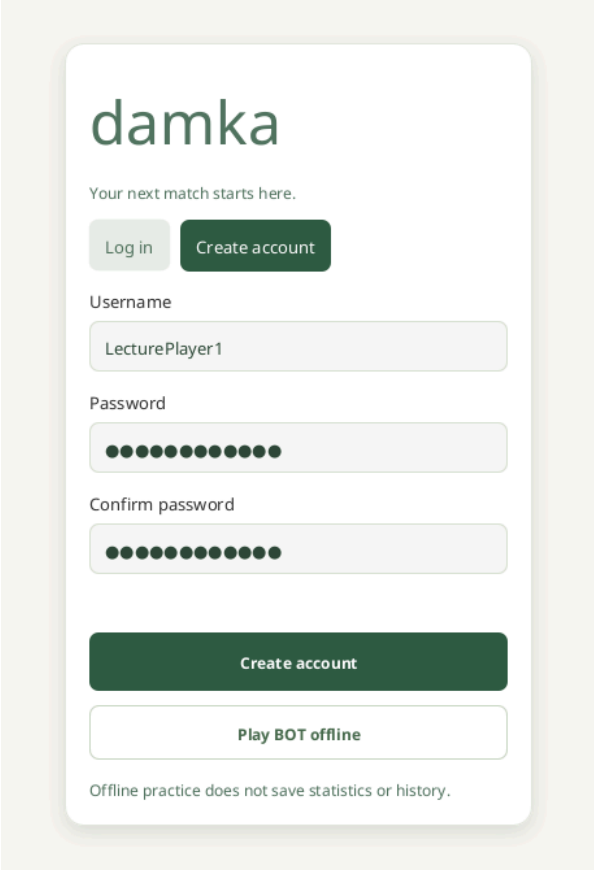
ProfileWindow מציגה את הנתונים. פתיחת החלון או Refresh שולחות PROFILE_REQUEST בלי שם משתמש לבחירה. השרת קורא את הפרופיל של החשבון המאומת, והלקוח מפענח באמצעות ProfileData.decode. סגירת חלון הפרופיל משאירה את חלון המשחק פתוח.

6.6 מתי תוצאת BOT נשמרת

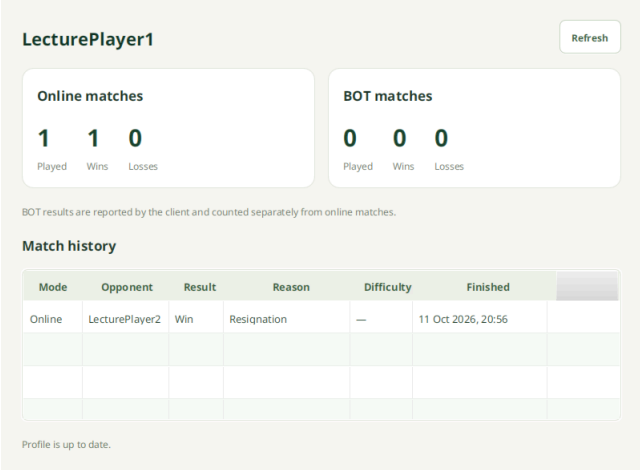
תוצאת BOT נשמרת כאשר המשחק מתחיל ומסתיים באותו חשבון פעיל ובאותו חיבור. המשחק מתנהל במחשב השחקן, והלקוח מדווח לשרת את התוצאה; היא מוצגת בנפרד ממשחקי הרשת. משחק ללא התחברות הוא אימון שאינו נכנס להיסטוריה, גם לאחר כניסה מאוחרת לחשבון.

7. ממשק המשתמש ומסכי המערכת

7.1 הזדהות ופרופיל



איור 6 — הרשמה בממשק הנוכחי.



איור 7 — פרופיל אחרי משחק הדגמה.

AccountView מציעה Log in, Create account ו־Play BOT offline. הצלחת הזדהות מובילה למסך המשחק. Profile פותח חלון עם שתי קבוצות סטטיסטיקה וטבלת היסטוריה; Refresh מבקש מידע חדש. הצילומים הופקו עם חשבונות הדגמה במסד זמני, ולא עם נתוני שחקנים אמיתיים.

7.2 בחירה, משוב ועיצוב

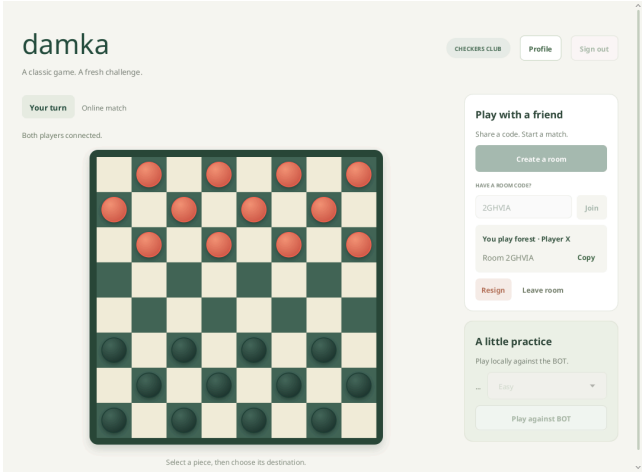
לחיצה על כלי בוחרת אותו, ולחיצה על יעד פנוי מבקשת מהלך. BoardView מציגה לוח קרם וירוק, כלי X בירוק וכלי O באלמוג. בחירה מסומנת בטבעת זהובה ומלך במסגרת עבה. שגיאה מופיעה בכרטיס צף שניתן לסגור; היא אינה מקטינה את הלוח, ופעולה מוצלחת מנקה הודעה קודמת. העיצוב נמצא ב־damka.css.

7.3 חלון גמיש

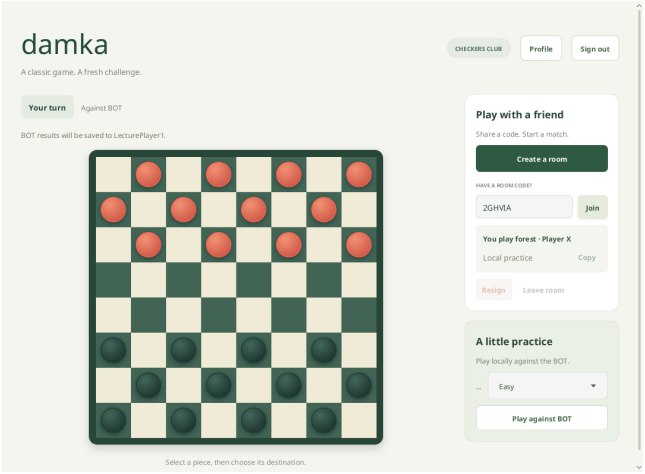
הלוח מתאים את גודלו לחלון. בחלון צר כרטיסי המשחק וה־BOT עוברים מתחתיו, וקיימת גלילה כשצריך. קוד החדר ניתן להעתקה באמצעות Copy. כתובת השרת והפורט מוגדרים בהפעלה ואינם שדות במסך.

7. ממשק המשתמש ומסכי המערכת — המשך

7.4 משחק רשת ומשחק BOT



איור 8 — חדר פעיל מול שחקן.



איור 9 — משחק BOT מקומי.

Create a room ו־Join משתמשות בחיבור המאומת. אחרי ששני המשתתפים מחוברים מוצגים הלוח והתור של השרת. Resign ו־Leave room מטפלות בסיום. Play against BOT מתחיל משחק מקומי; בוחרים Easy, Medium או Hard. הממשק מונע מהלך נוסף בזמן בקשה או אנימציה.

7.5 אותה אנימציה בשני המצבים

BoardView.animateMove מסתירה את הכלי במקור ומציגה כלי נע בשכבה מעל הלוח. Timeline מנפישה במשך כ־320 מילישניות; FadeTransition מעלימה כלי שנאכל. מרכזי התאים מחושבים בכל פריים כדי ששינוי גודל וגלילה לא יזיזו את מסלול התנועה. בסוף מוצגת התמונה המעודכנת. אותו רכיב משמש BOT ומשחק רשת.

7.6 היקף ההדגמה

הצילומים מציגים את זרימת השימוש ביישום: הזדהות, יצירת חדר, הצטרפות, ביצוע מהלכים וסיום משחק. בפרופיל מוצגת תוצאת משחק ההדגמה שנשמרה בשרת.

8. בדיקות ואימות המערכת

תחומי הבדיקה

בדיקות JUnit 5 בוחנות את מנוע המשחק, את התקשורת ואת שמירת הנתונים. בדיקות Game פועלות ישירות על המודל. בדיקות החדרים וההזדהות משתמשות בחיבורי TCP לכתובת loopback, ובדיקות הנתונים משתמשות במסדי SQLite זמניים. כך אפשר לחזור על הבדיקות בלי לשנות חשבונות ותוצאות של משתמשים.

מחלקת בדיקה	מספר בדיקות בהרצה המתועדת	תחום
GameRulesTest	11	חוקי משחק, רצף אכילות, הכתרה, העתקה וניצחון.
RoomProtocolTest	11	תורות, מהלכים, הודעות, הצטרפות וסיום חדר.
RoomConnectionTest	4	חיבור הלקוח, סדר הודעות וסגירה.
AuthProfileTest	14	הרשמה, כניסה, זהות, פרופיל ושמירת BOT.
ProfileDatabaseTest	8	חשבונות, התמדה, שמירת תוצאות ונתוני פרופיל.

תוצאות ובדיקת הממשק

בהרצה המתועדת מ־12.10.2026 עברו כל 48 הבדיקות והבנייה הסתיימה בהצלחה. נבדקו בין היתר אכילה מחייבת, רצף אכילות, הכתרה, תורות, הרשמה, כניסה ושמירת תוצאות ללא כפילות.

בנוסף לבדיקות האוטומטיות, הממשק נבחן ידנית ב־Windows באמצעות שרת ושני חלונות לקוח. התרחישים כללו יצירה והצטרפות לחדר, ביצוע מהלכים, הצגת הודעות ופתיחת פרופיל. המראה, התאמת החלון והאנימציה נבחנים בתוך JavaFX לצד בדיקות המודל והתקשורת.

8. בדיקות ואימות המערכת — המשך

תרחישים משמעותיים

תרחיש	תוצאה צפויה	אופן בדיקה
מהלך חוקי	אותו BOARD ותור מעודכן אצל שני הלקוחות.	בדיקת פרוטוקול עם שני חיבורי TCP.
מהלך מחוץ לתור או מהלך לא חוקי	ERROR למבקש ללא שינוי הלוח.	השוואת המצב לפני הבקשה ואחריה.
רצף אכילות	אותה חתיכה חייבת להמשיך וההתור נשאר אצלה.	בדיקות כללים ופרוטוקול.
סיסמה שגויה	הכניסה נדחית.	בדיקות הזדהות.
חשבון אחד בשני מקומות בחדר	הצטרפות למקום השני נדחית.	בדיקת זהות המשתתפים.
שליחת אותה תוצאה שוב	מספר המשחקים אינו גדל פעמיים.	בדיקת מזהה תוצאה וסטטיסטיקה.
פתיחה מחדש של המסד	החשבון והתוצאות עדיין קיימים.	בדיקות התמדה ב־SQLite.

9. הפעלה ומדריך משתמש

9.1 התחלת משחק ברשת

- פותחים את הפרויקט בסביבת הפיתוח ומפעילים את DamkaServer.
- כל שחקן מפעיל את Main ונרשם או נכנס לחשבון. לצורך הדגמה במחשב אחד פותחים שני חלונות.
- השחקן הראשון בוחר Create a room ומשתף את הקוד. השני מזין אותו ובוחר Join.
- כאשר שני השחקנים מחוברים, X מתחיל. בוחרים כלי ומשבצת יעד וממתינים לעדכון התור.

אפשר להפעיל את המחלקות באמצעות Run ב־VS Code לאחר ייבוא הפרויקט. במשחק בין שני מחשבים הם מחוברים לאותה רשת מקומית, ושני הלקוחות מוגדרים לכתובת ולפורט של השרת.

9.2 פרופיל וסיום משחק

Profile פותח את הסטטיסטיקה וההיסטוריה. Refresh מעדכן את הנתונים, וסגירת חלון הפרופיל מחזירה לחלון המשחק. Resign מאפשר לפרוש ו־Leave room לצאת מהחדר. לאחר הסיום אפשר ליצור משחק נוסף.

9.3 משחק מול BOT

Play against BOT מתחיל משחק מקומי ברמת Easy, Medium או Hard. לתרגול ללא שרת מפעילים רק Main ובוחרים Play BOT offline. משחק אימון אינו נרשם בפרופיל.

10. שימוש בבינה מלאכותית

בפרויקט נעשה שימוש נרחב ב־Codex ובּ־GitHub Copilot ללמידה ולתכנון, לכתיבת הקוד ולשינויו, לעיצוב הממשק ולהכנת בדיקות ותיעוד. הכלים סייעו בהסבר מושגים ובהבנת הקשר בין רכיבי המערכת.

תהליך העבודה כלל הגדרת דרישות, קבלת הצעות מכלי ה־AI, קריאת הקוד וההסברים, הרצה ובדיקות וביצוע התאמות בהתאם לתוצאות. הסיוע שימש הן לקידום המימוש והן ללמידה ולהבנת הפרויקט.

11. תרומה אישית

תחומי האחריות שהוגדרו בפרויקט:

סטודנט	תחום אחריות
רביע אבו עאד	לוגיקת המשחק (Game Logic).
מותנא פרחאת	חיבור TCP בין הלקוחות לשרת.

12. סיכום, אתגרים ומקורות

12.1 סיכום

הפרויקט מממש משחק דמקה במבנה שרת ולקוחות, עם חדרי משחק, חשבונות ופרופילים. השרת מנהל את המשחק המקוון, Game מרכזת את הכללים והלקוחות מציגים את המצב ומבקשים פעולות. אפשר לשחק גם מול BOT ולצפות בסטטיסטיקה ובהיסטוריה של החשבון.

12.2 אתגרים ופתרונות

- לימוד כללי הדמקה: לפני מימוש הכללים נלמדו התנועה, האכילה המחייבת, רצף האכילות וההכתרה, והוגדרו מצבי משחק לבדיקת המימוש.
- עבודה עם JavaFX: נדרשה היכרות עם ספריית ממשק חדשה, ארגון רכיבים בחלון וחיבור פעולות לכפתורים וללוח.
- יישום TCP: הידע התאורטי מקורס רשתות תקשורת יושם לראשונה בחיבור מעשי בין לקוחות לשרת ובהעברת הודעות משחק.
- ממשק יישום לעומת HTML: הניסיון בבניית דפי HTML שימש בסיס לחשיבה על עיצוב, והעבודה ב־JavaFX דרשה ללמוד כיצד לארגן רכיבי Java ולהגיב לאירועי משתמש.

12.3 המשך פיתוח

אפשר להוסיף צ׳אט בין המשתתפים בתוך חדר המשחק. הודעות הצ׳אט יוכלו לעבור דרך חיבורי TCP הקיימים אל השרת, שיעביר אותן לשחקן האחר באותו חדר. התוספת תאפשר לשחקנים להתכתב במהלך המשחק, לצד הלוח והתור.

12.4 מקורות

מאגר הפרויקט ב־GitHub:

https://github.com/MuthannaFrhat/damka-game